Introduction Of FoundationDB

ghostwydwu (武勇栋)

Tencent 腾讯 APD 架构平台部

Table Of Content

- history
- features
- architecture
- Key-Value Store
- SQL Layer
- summary

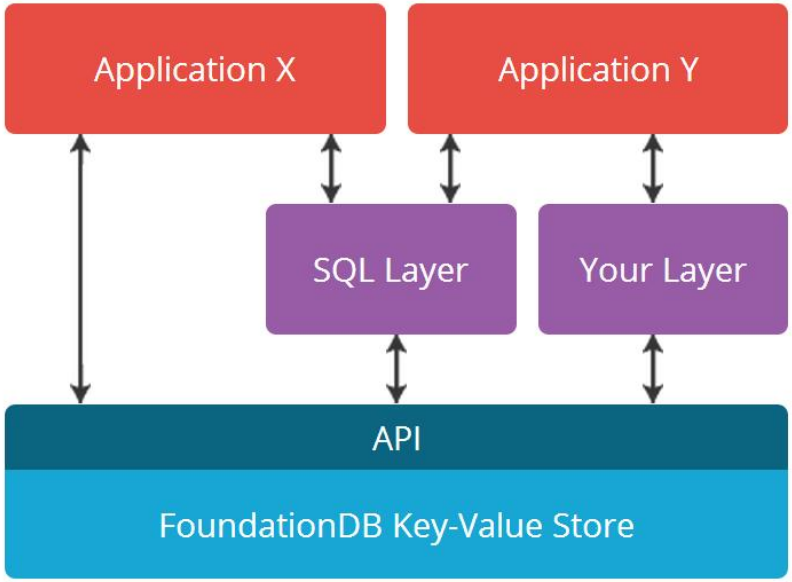
History

- Founded in 2009
- Two rounds of financing totaling $22.7 million
- Acquired by Apple in 2015

Features

- Multi－model
- Ultra－high scalability
- Supports ACID transactions across multiple data elements

Architecture



1. Your applications can send their data directly to the Key-Value Store or to a Layer.
2. A Layer can provide a new data model, compatibility with existing systems, or even serve as an entire framework.
3. **All data is stored in a single place** via an ordered, transactional Key-Value API.

APD 架构平台部

Key-Value Store

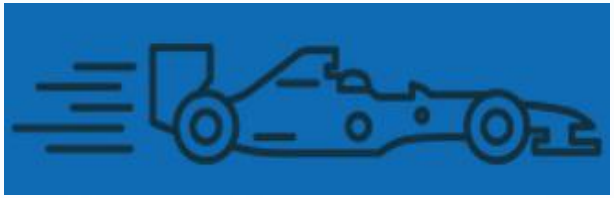
——The core of FoundationDB

Multi-model



The Key-Value Store's full multi-node ACID transactions allow developers to "layer" multiple data models on top of a single scalable database.

Scalable & fast



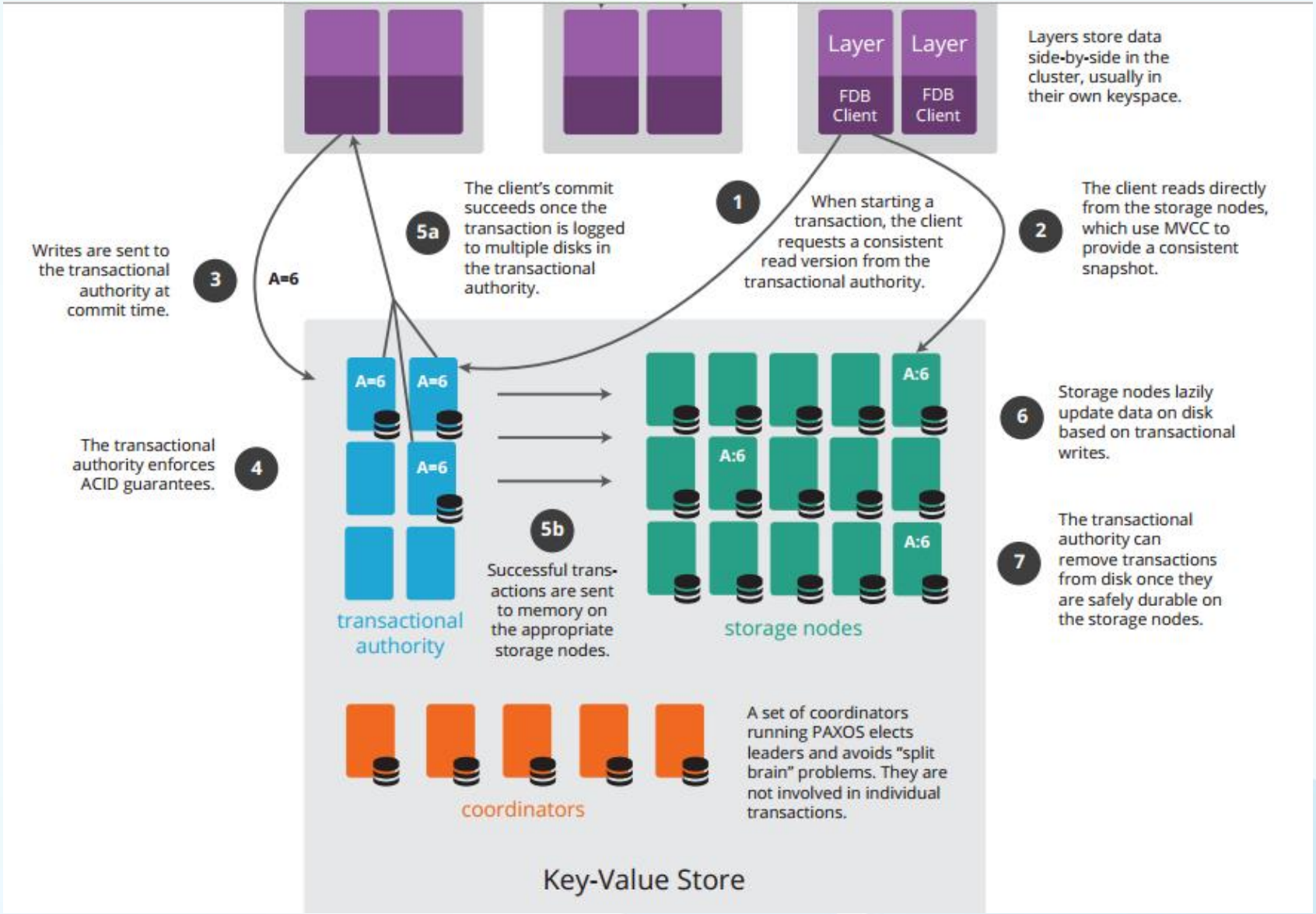
Key-Value Store provides amazing performance on commodity hardware and makes it incredibly easy to add more machines as your needs grow

Fault-tolerant



No more worrying about down time. Key-Value Stor divides your data into chunks and distributes multiple copies of each chunk onto different storag servers.

Key-Value Store



Key-Value Store – transaction -ACID

- all reads in the transaction see the same snapshot of the database
- the writes within a transaction either all succeed or all fail
- after a transaction succeeds (commits), the writes are permanently stored

Benefits of ACID in Key – Value Store

- - Transactions simplify concurrency control.
- - Non-intersecting transactions do not interfere with each other.
- - Transactions enable clean abstraction layers.
- - Merging modifications to data and indexes within one transaction makes layered database implementations feasible.
- - Transactions enable efficient data representations.

Transactions provide serializability. No explicit locks are required for data modification, which improves write-side performance.

Key-Value Store – transaction implemention

Public materials do not fully disclose FoundationDB's ACID implementation. Google MegaStore implements transaction serializability in a similar fashion, which is referenced below.

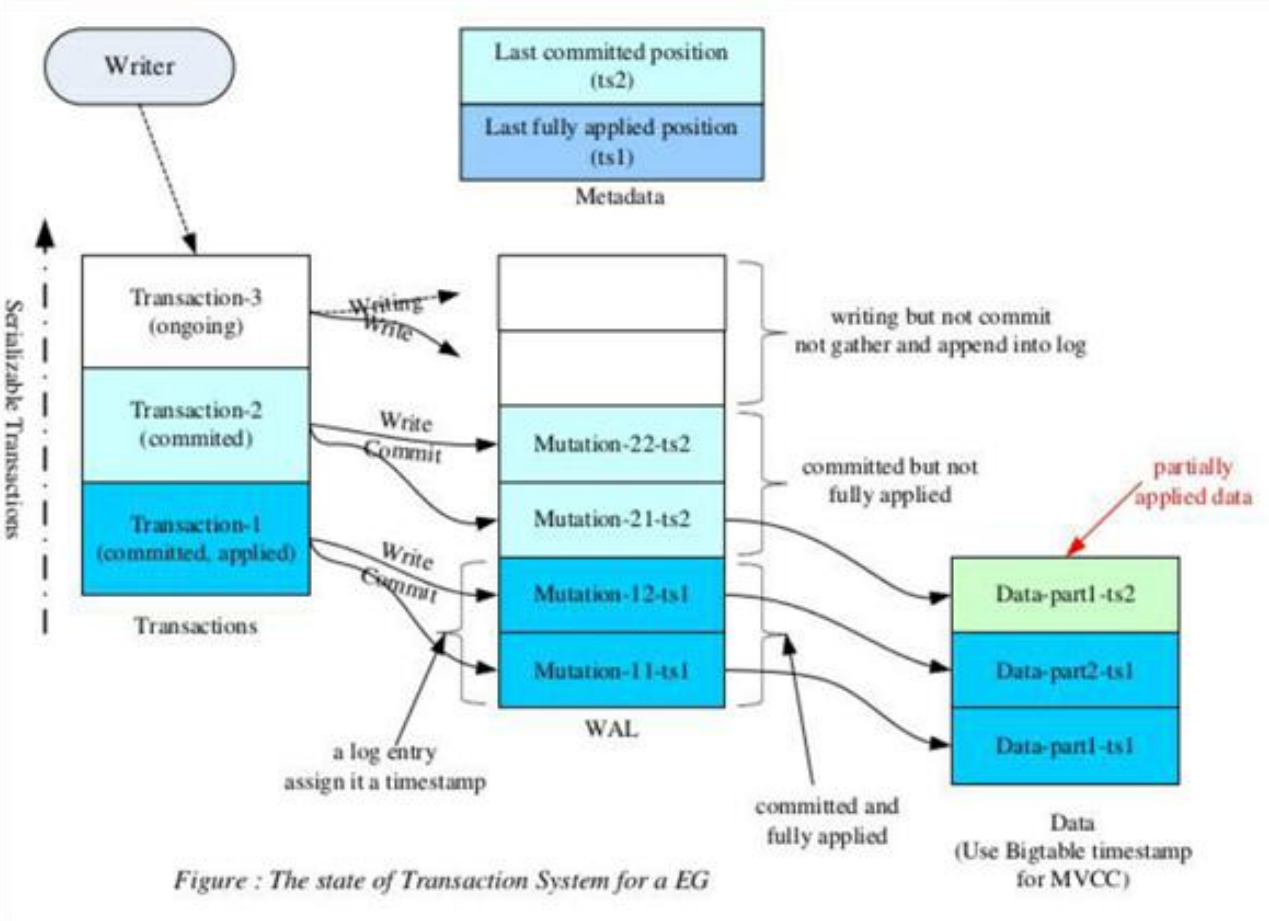


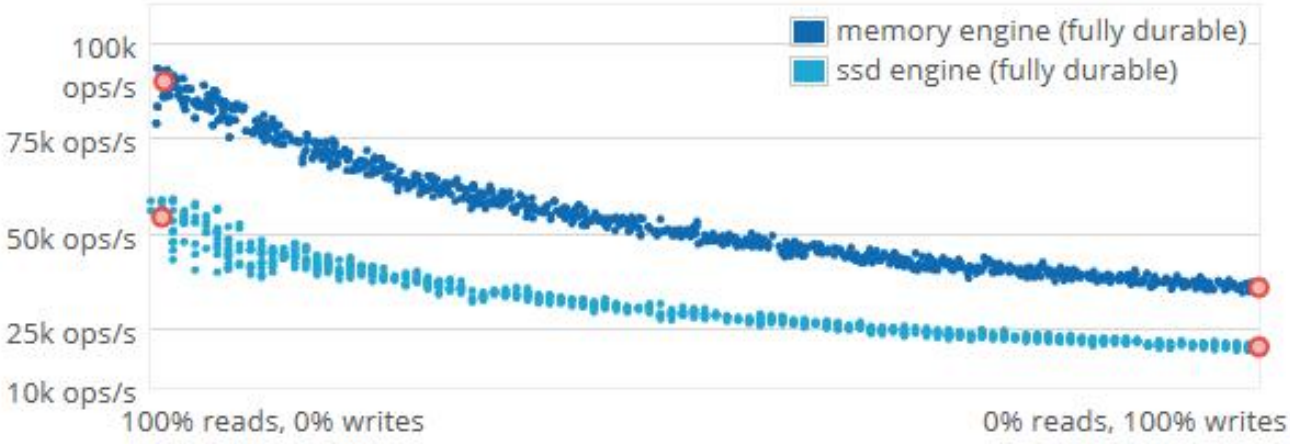
Figure : The state of Transaction System for a EG

APD 架构平台部

Key-Value Store Performance

Throughput (per core)

Key-Value Store provides good throughput for the full range of read and write workloads, with two fully durable storage engine options.

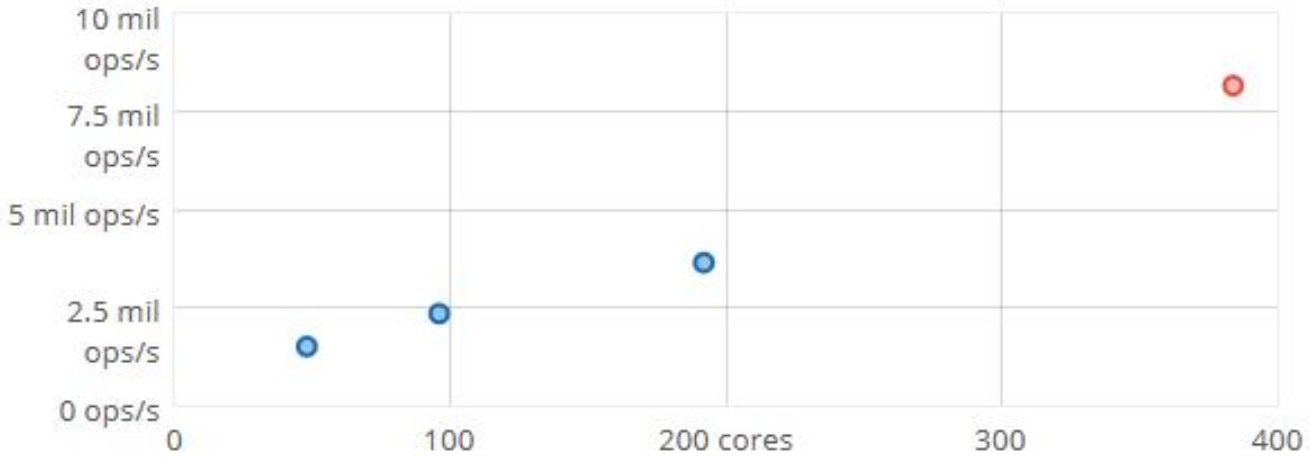


APD 架构平台部

Key-Value Store Performance

Scaling

Key-Value Store has an unmatched ability to scale linearly as you add more cores to a cluster.

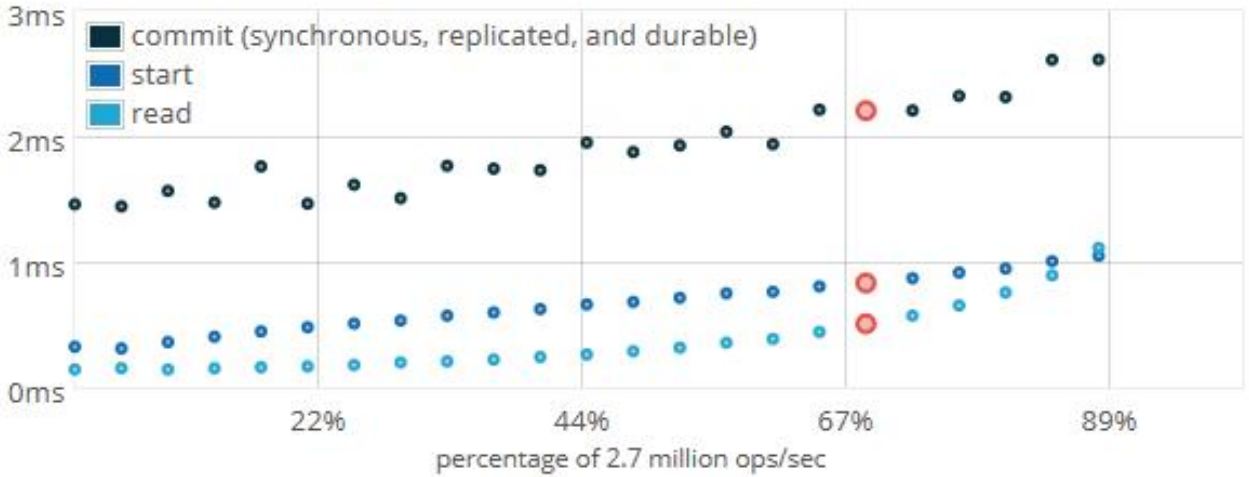


APD 架构平台部

Key-Value Store Performance

Latency

Key-Value Store has low latencies over a broad range of workloads that only increase modestly as the cluster approaches saturation.



APD 架构平台部

SQL Layer-Brief

1. Concept

   The SQL Layer is a fault tolerant and scalable RDBMS with distinct features aimed at making operational SQL truly effective in a distributed architecture. It is built as a Layer above the FoundationDB K/V store.

2. SQL Layer Features

3. Architecture

4. Table-Groups

5. Mapping SQL to K/V store

6. Known Limitations

SQL Layer--Features

1. **Highly available**
   The SQL Layer relies on and inherits from the Key-Value Store a transactional, scalable, highly available, and ordered storage substrate.

2. **Stateless scalability**
   All state is in the storage substrate, the SQL Layer retains no state of its own.

3. **Truly transactional**
   The SQL Layer is strongly consistent and supports true ACID properties even for complex transactions.

4. **Hierarchical schema:**
   a relational schema with explicit table hierarchies

5. **Sophisticated indexing**
   It includes both ordinary table indexes and also Table-Group multi-table indexes, which take columns from multiple tables.

SQL Layer--Features

6. Batch pipelining for parallel query execution

For example, say a query retrieves the last 500 users who logged into a website. A traditional RDBMS will operate serially:

```
scan the index of login_datetime in DESC order
for each index entry in the database
    extract the ID of the original row
    do a random access to retrieve the original row
    return to read the next index entry
```

```
scan the index of login_datetime in DESC order
for each index entry in the database
    extract the ID of the original row
    asynchronously request the original row
    return to read the next index entry
When an asynchronous request for a row returns
    Continue with that row to the next operator in the execution plan
```

SQL Layer--Features

7. Difference from a traditional RDBMS

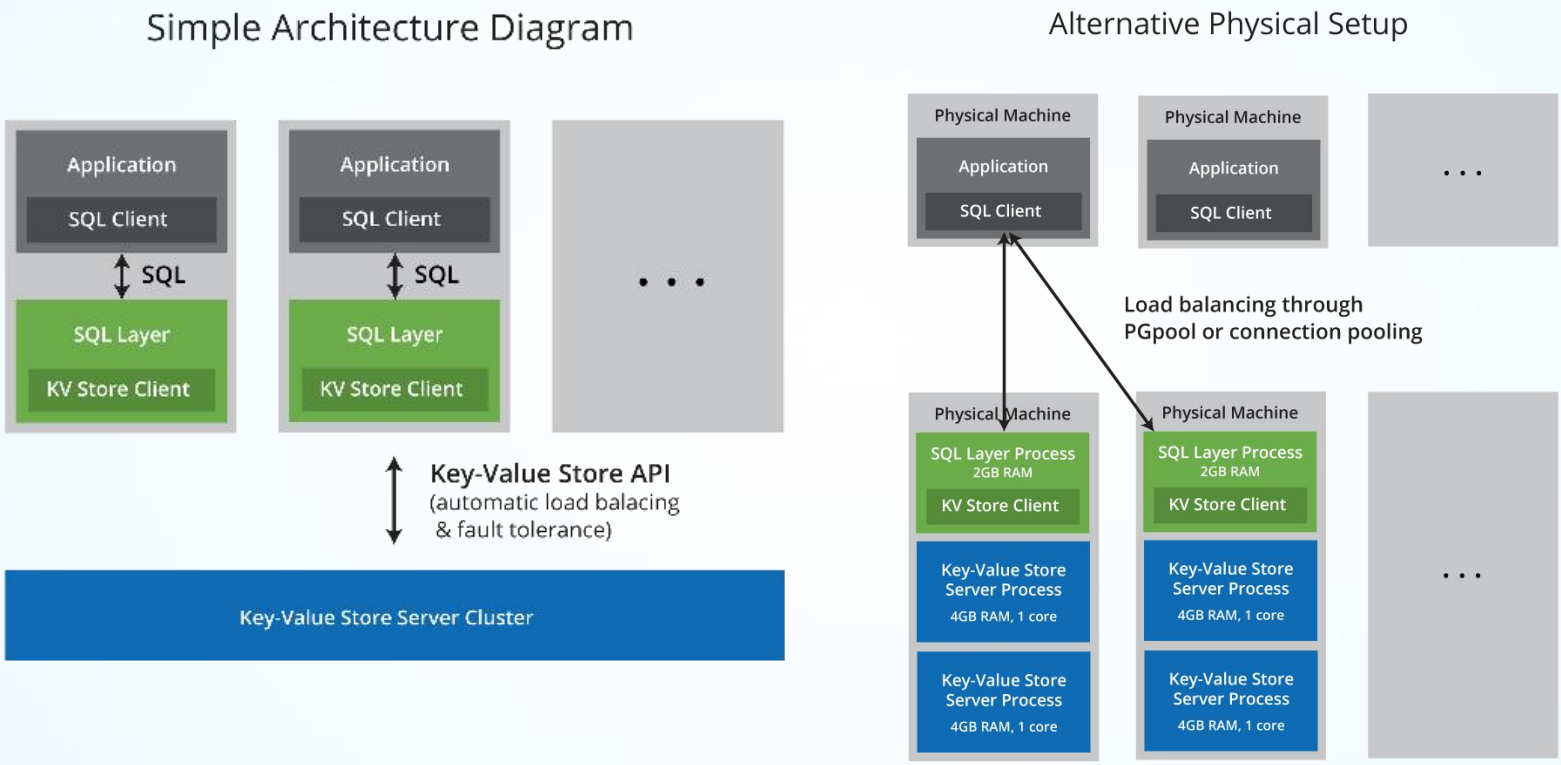
Differences from a traditional RDBMS

The following table summarizes the main differences:

Feature	Traditional RDBMS	SQL Layer
Scaling	Scale up (i.e. larger machine) or sharding	Scale out by adding nodes at the K/V level and the SQL level (see Architecture)
Fault tolerance	Use replication and configure fail-over mechanism by hand	Any cluster of 5 or more K/V nodes has strong fault tolerance
Query performance	Dependent on total data accessed, good as long as most data is in memory	Batch pipelining to parallelize query execution
Query timeout	Configurable	Up to 5 seconds with full isolation
Latency	Low for in-memory data, about 0.1ms for SSD and a few milliseconds for HDD	A few milliseconds
Throughput	Limited to saturating all cores	Linear scalability with number of nodes & cores
Transaction Isolation	Configurable, usually defaults to `READ COMMITTED`	`SERIALIZABLE`, `SNAPSHOT`, `READ COMMITTED NO SNAPSHOT`
Transaction timeout	Configurable	Up to 5 seconds

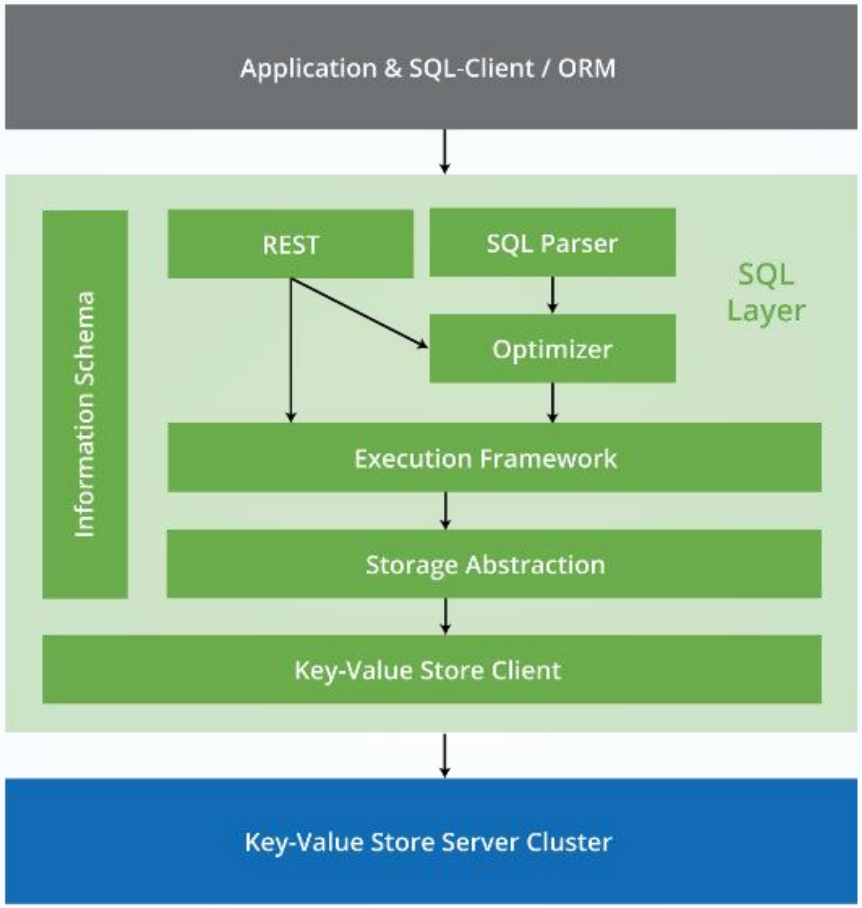
SQL Layer--Architecture

1. Architecture



APD 架构平台部

SQL Layer—Inner Architecture



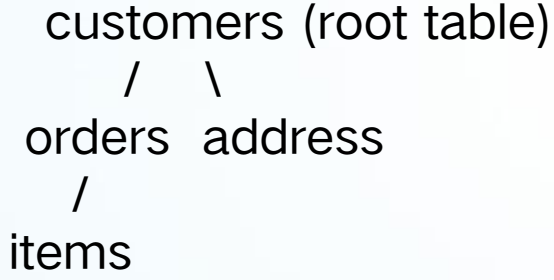
APD 架构平台部

SQL Layer-Tablet Groups

1. What is Tablet Groups

In essence, a Table-Group represents an object as a nested, hierarchical set of rows.

Table-Groups organize a set of user-defined “relational” tables into an explicit table hierarchy that reflects an object structure



SQL Layer-Tablet Groups

2. Clustered storage

The Table-Group definition is reflected in the physical organization of data. Table-Groups define the way physical data is stored and deeply impact the way in which the SQL Layer operates.

```
customer row (Key:C1)
   order row (Key:C1,O20)
    item row (Key:C1,O20,I300)
    item row (Key:C1,O20,I301)
    item row (Key:C1,O20,I302)
   order row (Key:C1,O21)
    item row (Key:C1,O21,I303)
    item row (Key:C1,O21,I304)
 address row (Key:C1,A17)
 address row (Key:C1,A18)
customer row (Key:C2)
   order row (Key:C2,O22)
    item row (Key:C2,O22,I305)
 address row (Key:C2,A18)
 ...
```

hkey	row
c1	c1, 'ori'
c1o1	o1c1, '2010/1/1'
c1o1i1	i1, o1, $10.00, 5
c1o1i2	i2, o1, $20.00, 10
c1o2	o2, c1, '2010/3/15'
c1o2i3	i3, o2, $5.00, 100
c1o2i4	i4, o2, $17.00, 3
c1a1	a1, c1, 101 Main Street, Cambridge MA
c2	c2, 'david'
c2o3	o3, c2, '2010/6/4'
c2o3i5	i5, o3, $50.00, 8
c2a2	a2, c2, Corner, VA
c2a3	a3, c2, Cape Cod, MA

SQL Layer- Mapping SQL data to the Key-Value Store

1. **How does the SQL Layer store the data in the Key-Value store?**

The SQL Layer stores metadata and actual data (e.g. table content) in the Key-Value Store.

Directory	Tuple	Raw Key
sql /	(9)	\x15\x09
sql / data /	(3)	\x15\x03
sql / data / table /	(31)	\x15\x1F
sql / data / table / schema_a /	(228)	\x15\xE4
sql / data / table / schema_a / table_name_1 /	(215)	\x15\xD7
sql / data / table / schema_a / table_name_2 /	(247)	\x15\xF7

When using the tuple storage format, a row is stored as a single key-value pair. The key is a tuple formed by concatenating the directory prefix, the position within the Table-Group and the PRIMARY KEY. The value is a tuple formed from all of the columns in the row.

For example, the following inserts to the tables from the previous example would yield the given keys and values:

```
INSERT INTO schema_a.table_name_1 VALUES (1, 'hello'), (2, 'world');
INSERT INTO schema_a.table_name_2 VALUES (5);
```

Raw Key	Tuple Key	Raw Value	Tuple Value
\x15\xD7\x15\x01\x15\x01	(215, 1, 1)	\x15\x01\x02hello\x00	(1, 'hello')
\x15\xD7\x15\x01\x15\x02	(215, 1, 2)	\x15\x02\x02world\x00	(2, 'world')
\x15\xF7\x15\x01\x15\x05	(247, 1, 5)	\x15\x05	(5)

SQL Layer- Mapping SQL data to the Key-Value Store

2. How are table indexes stored by the SQL layer?

An index is located in the directory of the table it belongs, in a subdirectory with the name of the index.

3. How are groups stored by the SQL layer?

```
CREATE TABLE schema_a.table_name_3(id INT PRIMARY KEY, id_1 INT, GROUPING FOREIGN KEY (id_1) REFERENCES schema_a.table_name_1(id));
INSERT INTO schema_a.table_name_3 VALUES (100, 2), (200, 2), (300, 1);
```

This yields the following results:

```
directory = fdb.directory.open(db, ('sql','data','table','schema_a','table_name_1'))
for key, value in db[directory.range()]:
    print fdb.tuple.unpack(key), ' --> ', fdb.tuple.unpack(value)


(215, 1, 1)          -->  (1, u'hello')
(215, 1, 1, 2, 300)  -->  (300, 1)
(215, 1, 2)          -->  (2, u'world')
(215, 1, 2, 2, 100)  -->  (100, 2)
(215, 1, 2, 2, 200)  -->  (200, 2)
```

SQL Layer- Known Limitations

1. **Design Limitation**
   - 1.1 Long transactions and Large transactions
   - 1.2 Per-query latency

2. **Current limitations**
   - 2.1 Long-running Queries
   - 2.2 Single thread, single node query execution
   - 2.3 Cluster size -- 96 cores/processes and 48 SSDs on 24 machines
   - 2.4 Large keys and values – key:10KB, value:100KB
   - 2.5 Unsupported SQL syntax – （DESC, rename index,rename column……）
   - 2.6 System limitation
     - Max number of tables: 2^32
     - Max number of tables per schema: a few thousand
     - Max key size of indexes: 2,047 bytes
     - Max row size: 100,000 bytes
     - Max transaction timeout: 5 seconds